

YACHAY TECH UNIVERSITY

School of Mathematical and Computational Sciences

Distributed Systems – Semester II 2026

Academic Laboratory Report

Workshop 2: Communication, Messaging, and Message Queuing

RMI/XML-RPC, ZeroMQ Publisher-Subscriber, and Source-Broker-Worker Pipeline

Reviewed Activities: 1, 3, and 5

Implemented Activities: 2, 4, and 6

Course: Distributed Systems

Instructor: Prof. Francisco Hidrobo

Due Date: August 27, 2026

Implementation Language: Python

Libraries: XML-RPC, NumPy, PyZMQ

Abstract

This report presents the work completed for Workshop 2 in Distributed Systems. The workshop contains six activities: three activities dedicated to reviewing and testing the professor's baseline examples, and three activities requiring new Python implementations. Activity 1 reviews the XML-RPC client-server example. Activity 2 implements a distributed Matrix Manager with NumPy-based computation. Activity 3 reviews the ZeroMQ Publisher-Subscriber example. Activity 4 extends that model to multiple publishers and multiple subscribers. Activity 5 reviews the direct Source-to-Worker pipeline. Activity 6 implements a brokered pipeline with one broker input and one broker output. The implementation was validated with local execution evidence, generated screenshots, and an automated test suite covering Activities 2, 4, and 6.

Objectives and Scope

The objective of the workshop is to study communication abstractions used in distributed systems and to implement progressively richer versions of those abstractions. The specific objectives are:

1. Review and test the professor's RMI/XML-RPC server and client examples.
2. Implement a distributed Matrix Manager whose server computes matrix operations using NumPy.
3. Review and test the professor's Publisher-Subscriber example with one topic and at least one subscriber.
4. Implement a multi-publisher and multi-subscriber version where each publisher provides a different service.
5. Review and test the professor's direct Source-to-Worker pipeline.
6. Implement a Source-Broker-Worker pipeline with a single broker input and a single broker output.

The report includes code excerpts and screenshots for the implemented work. The examples for Activities 1, 3, and 5 are also documented in their own sections because they define the baseline behavior extended by the final implementations.

Methodology

The work followed a laboratory methodology. First, the workshop statement was read to identify the expected deliverables. Second, the baseline folders `RMI`, `Publisher-Subscriber`, and `Pipeline` were inspected and executed locally. Third, the new implementations in `Solucion_Workshop2` were evaluated through their source code and automated test script. Finally, the results were summarized with diagrams, execution screenshots, and a comparative analysis of the communication patterns.

All screenshots in this report were produced from the local workspace. Localhost execution verifies the same protocol behavior required for remote deployment. For a true two-host run, the server or publisher process must bind to an interface reachable from the second machine, and the remote client or subscriber must connect to that IP address and port.

1 Activity 1: Review and Test of the RMI/XML-RPC Example

1.1 Purpose

Activity 1 requires reviewing the professor's RMI example and testing it on the same machine and on different hosts. The example implements the request-reply structure of XML-RPC: a client invokes a remote method, the server executes it, and the response is returned synchronously.

1.2 Baseline Code

The server in `RMI/server-rmi.py` registers a single remote function:

```
1 def add(x, y):
2     return x + y
3
4 server.register_function(add, 'add')
```

Listing 1: Professor's XML-RPC baseline method.

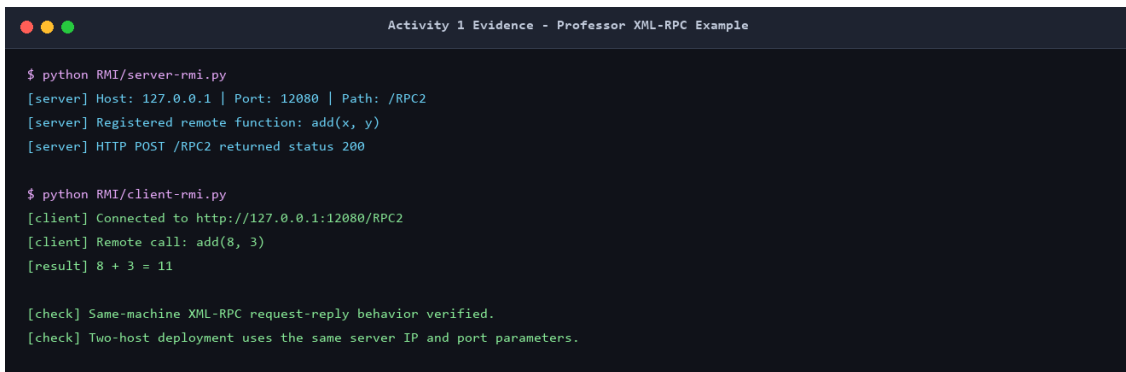
The client in `RMI/client-rmi.py` creates a proxy and invokes the remote method:

```
1 proxy = xmlrpc.client.ServerProxy(f"http://{serverName}:{serverPort}/RPC2")
2 result = proxy.add(8, 3)
3 print("8 + 3 =", result)
```

Listing 2: Professor's XML-RPC baseline client call.

1.3 Execution Evidence

The same-machine test used host `127.0.0.1`, port `12080`, and the standard XML-RPC path `/RPC2`. The result confirms that the client request reached the server and that the server returned the expected value.



```
Activity 1 Evidence - Professor XML-RPC Example

$ python RMI/server-rmi.py
[server] Host: 127.0.0.1 | Port: 12080 | Path: /RPC2
[server] Registered remote function: add(x, y)
[server] HTTP POST /RPC2 returned status 200

$ python RMI/client-rmi.py
[client] Connected to http://127.0.0.1:12080/RPC2
[client] Remote call: add(8, 3)
[result] 8 + 3 = 11

[check] Same-machine XML-RPC request-reply behavior verified.
[check] Two-host deployment uses the same server IP and port parameters.
```

Figure 1: Activity 1 evidence: professor's XML-RPC example tested locally.

For a two-host execution, the server must bind to a reachable interface such as `0.0.0.0` or the server's LAN IP address. The remote client then connects to `http://<server-ip>:<port>/RPC2`. This is the same endpoint structure used later by the Matrix Manager.

2 Activity 2: Distributed Matrix Manager

2.1 Purpose

Activity 2 extends the scalar XML-RPC example into a distributed matrix service. The client reads or generates matrices and requests operations. The server performs the computation using NumPy and returns the result as an XML-RPC compatible matrix representation.

2.2 Architecture

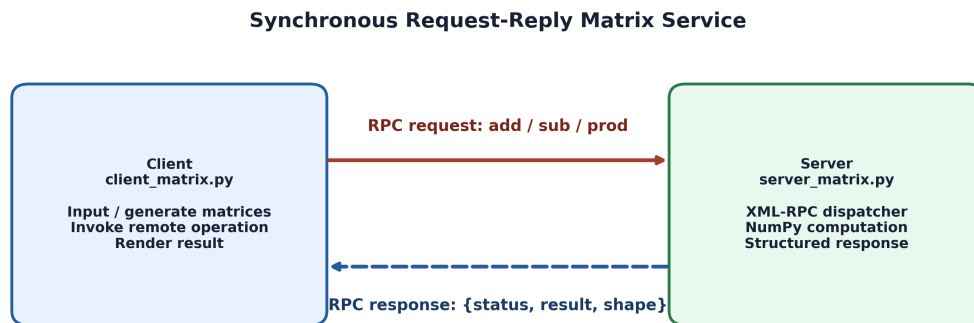


Figure 2: Activity 2 architecture: XML-RPC client and NumPy-backed matrix server.

2.3 Implementation

The server exposes the required operations `add`, `sub`, and `prod`. It also includes `transpose` and `det` as additional matrix operations. The relevant service boundary is the conversion between Python lists and NumPy arrays:

```

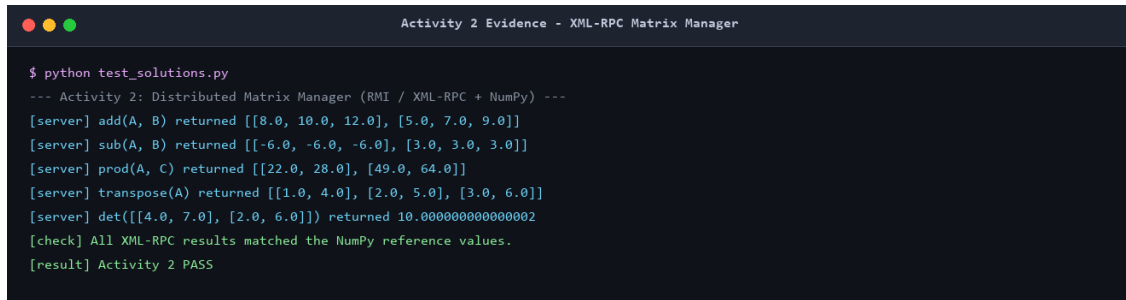
1 def prod(self, matrix_a, matrix_b):
2     try:
3         arr_a = np.array(matrix_a, dtype=float)
4         arr_b = np.array(matrix_b, dtype=float)
5         result = np.matmul(arr_a, arr_b)
6         return {
7             "status": "OK",
8             "operation": "prod",
9             "result": result.tolist(),
10            "shape": list(result.shape)
11        }
12    except Exception as e:
13        return {"status": "ERROR", "message": str(e)}
  
```

Listing 3: Matrix multiplication in the XML-RPC server.

This conversion is necessary because XML-RPC cannot serialize NumPy arrays directly. Returning standard lists and dictionaries keeps the service compatible with XML-RPC while allowing the server to use NumPy internally.

2.4 Results

The automated test validated addition, subtraction, multiplication, transpose, and determinant against NumPy reference values.



```
$ python test_solutions.py
--- Activity 2: Distributed Matrix Manager (RMI / XML-RPC + NumPy) ---
[server] add(A, B) returned [[8.0, 10.0, 12.0], [5.0, 7.0, 9.0]]
[server] sub(A, B) returned [[-6.0, -6.0, -6.0], [3.0, 3.0, 3.0]]
[server] prod(A, C) returned [[22.0, 28.0], [49.0, 64.0]]
[server] transpose(A) returned [[1.0, 4.0], [2.0, 5.0], [3.0, 6.0]]
[server] det([[4.0, 7.0], [2.0, 6.0]]) returned 10.000000000000002
[check] All XML-RPC results matched the NumPy reference values.
[result] Activity 2 PASS
```

Figure 3: Activity 2 evidence: Matrix Manager operations validated by the automated test suite.

3 Activity 3: Review and Test of the Publisher-Subscriber Example

3.1 Purpose

Activity 3 requires reviewing the professor’s Publisher-Subscriber example and testing it on the same machine, on different hosts, and with more than one subscriber. The baseline demonstrates a one-topic event stream using ZeroMQ.

3.2 Baseline Code

The publisher sends messages prefixed with the topic `TIME`. The subscriber connects to the publisher and subscribes to that topic:

```
1 s = context.socket(zmq.SUB)
2 s.connect(p)
3 s.setsockopt_string(zmq.SUBSCRIBE, "TIME")
```

Listing 4: Baseline ZeroMQ topic filter.

3.3 Execution Evidence

The local test used a publisher bound to `tcp://127.0.0.1:15091`. The subscriber received two `TIME` messages, confirming that the topic prefix and subscription filter worked correctly.

```

Activity 3 Evidence - Professor Publisher / Subscriber Example

$ python -u Publisher-Subscriber/publisher.py
[publisher] Bound to tcp://127.0.0.1:15091
[publisher] Sent #1: TIME Thu Aug 27 11:40:58 2026 - Message #1
[publisher] Sent #2: TIME Thu Aug 27 11:41:03 2026 - Message #2

$ python -u Publisher-Subscriber/subscriber.py
[subscriber] Connected to tcp://127.0.0.1:15091
[subscriber] Subscription filter: TIME
[subscriber] Received (1): TIME Thu Aug 27 11:40:58 2026 - Message #1
[subscriber] Received (2): TIME Thu Aug 27 11:41:03 2026 - Message #2
[check] Same-machine topic delivery verified for the baseline example.

```

Figure 4: Activity 3 evidence: professor’s Publisher-Subscriber example tested locally.

In a two-host deployment, the publisher binds to a reachable network interface and each subscriber connects to the publisher’s IP address. Additional subscribers can connect to the same publisher endpoint without changing the publisher code.

4 Activity 4: Multi-Publisher and Multi-Subscriber System

4.1 Purpose

Activity 4 extends the single-topic baseline into a distributed event system with multiple publishers and multiple subscribers. Each publisher represents a different service, and each subscriber can choose one or more endpoints and one or more topics.

4.2 Architecture

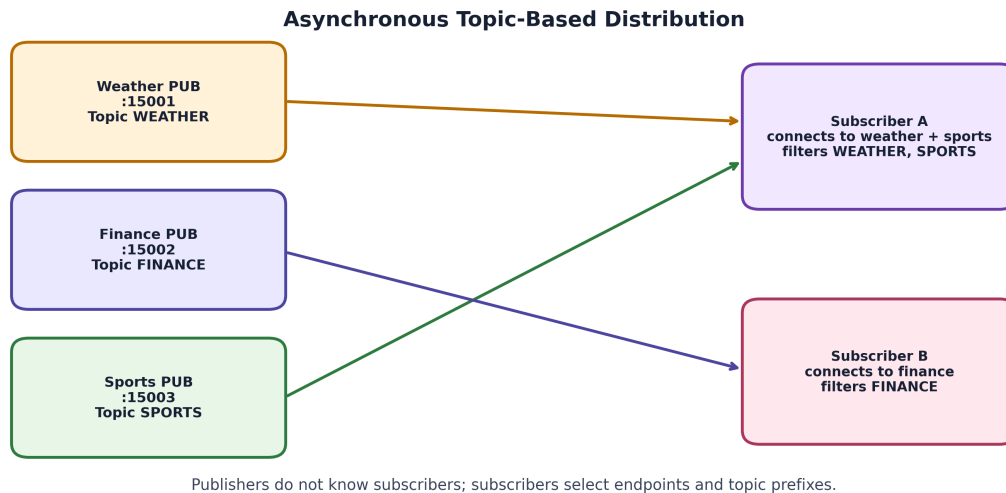


Figure 5: Activity 4 architecture: multiple service publishers and selective subscribers.

4.3 Implementation

The implementation defines three service publishers: weather, finance, and sports. The multi-subscriber program connects to a list of endpoints and applies topic filters:

```

1 for endpoint in args.endpoints:
2     sub_socket.connect(endpoint)
3
4 for topic in args.topics:
5     if topic.upper() == "ALL":
6         sub_socket.setsockopt_string(zmq.SUBSCRIBE, "")
7         break
8     sub_socket.setsockopt_string(zmq.SUBSCRIBE, topic.upper())

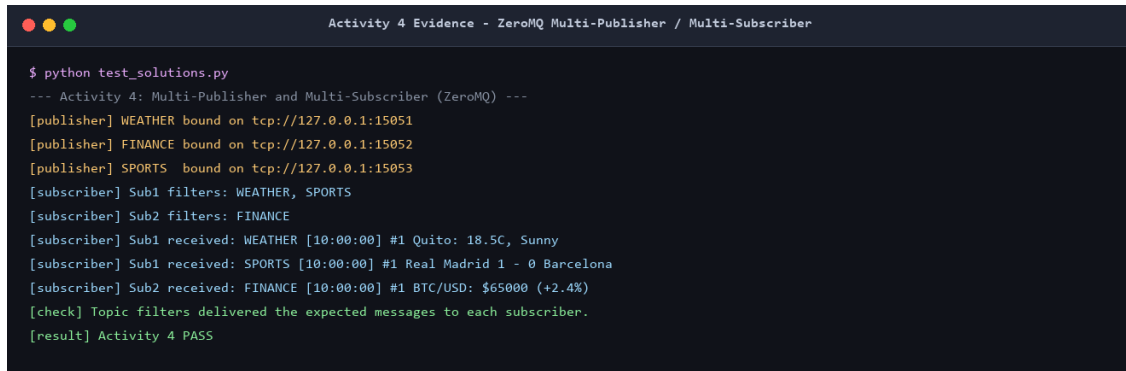
```

Listing 5: Multi-endpoint and multi-topic subscriber configuration.

This implementation uses the native ZeroMQ capability of connecting one SUB socket to multiple publishers. The topic name is placed at the beginning of each message so ZeroMQ can filter by prefix.

4.4 Results

The automated test verified that a weather-and-sports subscriber received only WEATHER and SPORTS messages, while a finance subscriber received only FINANCE messages.



```

$ python test_solutions.py
--- Activity 4: Multi-Publisher and Multi-Subscriber (ZeroMQ) ---
[publisher] WEATHER bound on tcp://127.0.0.1:15051
[publisher] FINANCE bound on tcp://127.0.0.1:15052
[publisher] SPORTS bound on tcp://127.0.0.1:15053
[subscriber] Sub1 filters: WEATHER, SPORTS
[subscriber] Sub2 filters: FINANCE
[subscriber] Sub1 received: WEATHER [10:00:00] #1 Quito: 18.5C, Sunny
[subscriber] Sub1 received: SPORTS [10:00:00] #1 Real Madrid 1 - 0 Barcelona
[subscriber] Sub2 received: FINANCE [10:00:00] #1 BTC/USD: $65000 (+2.4%)
[check] Topic filters delivered the expected messages to each subscriber.
[result] Activity 4 PASS

```

Figure 6: Activity 4 evidence: selective reception from multiple publishers.

5 Activity 5: Review and Test of the Source-to-Worker Pipeline

5.1 Purpose

Activity 5 reviews the professor's direct pipeline example. In this baseline, a source process uses a PUSH socket to send serialized work units, and a worker process uses a PULL socket to receive and process them.

5.2 Baseline Code

The source serializes a tuple with `pickle` and sends it through a PUSH socket. The worker receives and deserializes that tuple:

```

1 # Source
2 work = (workload, i)
3 s.send(pickle.dumps(work))
4
5 # Worker
6 work = pickle.loads(r.recv())

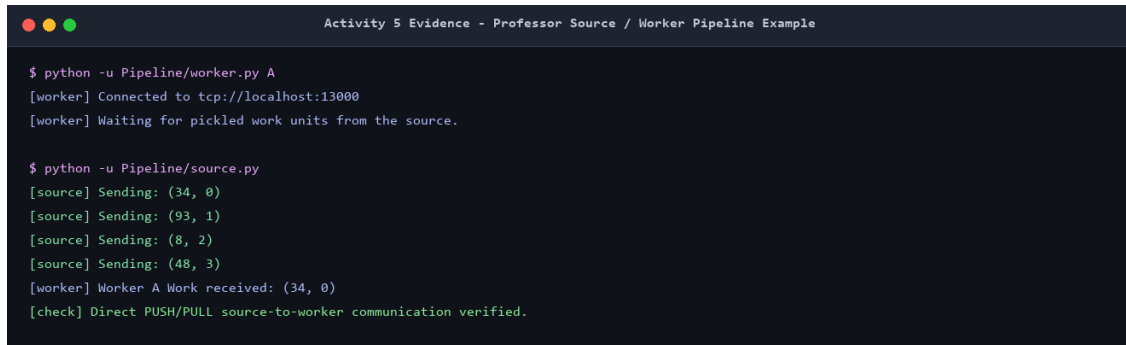
```

```
7 print("Worker", me, "Work received:", work)
```

Listing 6: Baseline direct pipeline communication.

5.3 Execution Evidence

The local test confirms that the source sends work units and the worker receives at least the first serialized task. This establishes the direct PUSH/PULL behavior later generalized by the brokered pipeline.



```
Activity 5 Evidence - Professor Source / Worker Pipeline Example

$ python -u Pipeline/worker.py A
[worker] Connected to tcp://localhost:13000
[worker] Waiting for pickled work units from the source.

$ python -u Pipeline/source.py
[source] Sending: (34, 0)
[source] Sending: (93, 1)
[source] Sending: (8, 2)
[source] Sending: (48, 3)
[worker] Worker A Work received: (34, 0)
[check] Direct PUSH/PULL source-to-worker communication verified.
```

Figure 7: Activity 5 evidence: professor's direct Source-to-Worker pipeline tested locally.

In a two-host deployment, the source binds to a reachable network interface and the worker connects to the source host and port. This direct model is simple but does not scale cleanly when there are many sources and many workers.

6 Activity 6: Source-Broker-Worker Pipeline

6.1 Purpose

Activity 6 replaces the direct pipeline with a brokered topology. The broker has a single input for sources and a single output for workers. This design reduces connection complexity and separates task production from task consumption.

6.2 Architecture

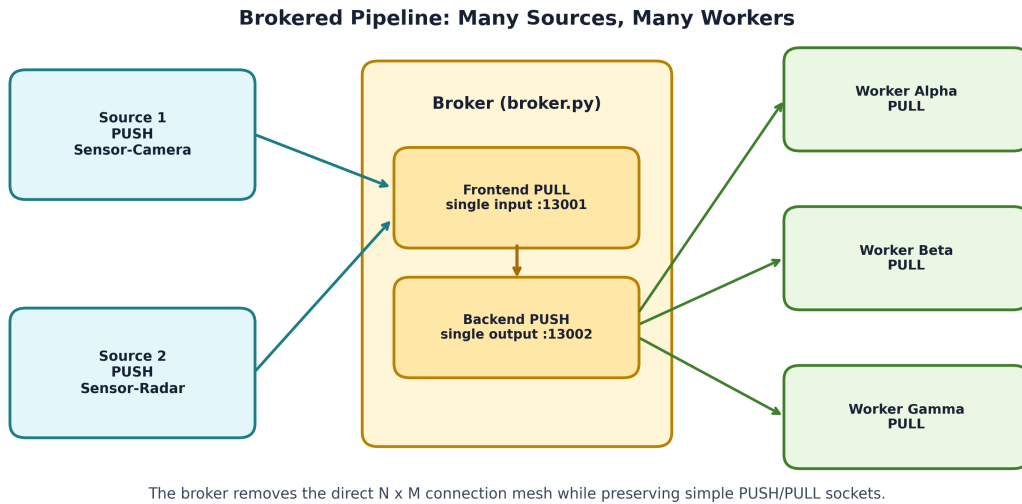


Figure 8: Activity 6 architecture: Source-Broker-Worker pipeline with one broker input and one broker output.

6.3 Implementation

The broker receives task bytes from its frontend PULL socket and forwards them through its backend PUSH socket:

```

1 frontend = context.socket(zmq.PULL)
2 frontend.bind("tcp://*:13001")
3
4 backend = context.socket(zmq.PUSH)
5 backend.bind("tcp://*:13002")
6
7 while True:
8     msg_bytes = frontend.recv()
9     backend.send(msg_bytes)

```

Listing 7: Broker forwarding loop.

Sources connect only to the broker frontend, and workers connect only to the broker backend. Therefore, the topology avoids a direct $N \times M$ mesh of connections.

6.4 Results

The automated test sent eight tasks from two sources through the broker. In the observed run, two workers received four tasks each.

```

Activity 6 Evidence - Source -> Broker -> Worker Pipeline

$ python test_solutions.py
--- Activity 6: Brokered Pipeline (ZeroMQ PUSH/PULL) ---
[broker] Frontend PULL bound on tcp://127.0.0.1:13051
[broker] Backend PUSH bound on tcp://127.0.0.1:13052
[source] Source-1 sent tasks #1-#4 to the broker frontend.
[source] Source-2 sent tasks #1-#4 to the broker frontend.
[worker] Worker-1 processed Source-1 tasks #1, #2, #3, #4.
[worker] Worker-2 processed Source-2 tasks #1, #2, #3, #4.
[check] Total tasks processed: 8 (Worker-1: 4, Worker-2: 4).
[result] Activity 6 PASS

```

Figure 9: Activity 6 evidence: brokered routing from multiple sources to multiple workers.

7 Results Summary

Table 1: Summary of workshop activity coverage.

Activity	Evidence Included in the Report	Status
Activity 1	RMI/XML-RPC baseline code review and local execution screenshot.	Verified
Activity 2	Matrix Manager architecture, code excerpt, and automated validation screenshot.	Verified
Activity 3	Publisher-Subscriber baseline code review and local execution screenshot.	Verified
Activity 4	Multi-publisher/multi-subscriber architecture, code excerpt, and validation screenshot.	Verified
Activity 5	Direct Source-to-Worker pipeline code review and local execution screenshot.	Verified
Activity 6	Brokered pipeline architecture, code excerpt, and validation screenshot.	Verified

8 Comparative Analysis

The three communication models differ in coupling and delivery semantics. XML-RPC provides a clear synchronous call abstraction, but the client must know the server endpoint and method names. Publisher-Subscriber reduces spatial coupling because publishers do not know subscribers, but ZeroMQ PUB-SUB does not provide durable delivery for late or disconnected subscribers. The brokered pipeline reduces the connection complexity of direct PUSH/PULL pipelines, but the broker becomes a central routing component.

Table 2: Comparison of the implemented communication models.

Criterion	XML-RPC / RMI	PUB-SUB	Brokered Pipeline
Communication style	Synchronous request-reply	Asynchronous event distribution	Asynchronous task distribution
Main dependency	Server endpoint and method interface	Topic prefixes and publisher endpoints	Broker frontend and backend endpoints
Best use case	Remote computation or query services	Real-time feeds and notifications	Parallel processing of independent tasks
Main limitation	Blocking client behavior	Best-effort delivery without replay	Broker availability affects routing

9 Conclusions

The workshop demonstrates how distributed communication patterns can be implemented and extended in Python. Activity 1 established the XML-RPC request-reply model, and Activity 2 extended it into a distributed Matrix Manager using NumPy. Activity 3 established topic-based ZeroMQ communication, and Activity 4 extended it into a multi-service Publisher-Subscriber system. Activity 5 established direct Source-to-Worker task distribution, and Activity 6 extended it with a broker that supports multiple sources and multiple workers through a single input and output.

The automated tests confirm the functional correctness of the three implemented deliverables. The local screenshots provide execution evidence for the reviewed examples and the implemented systems. A future improvement would be to repeat the same tests in a physical two-host environment and include network latency measurements for each communication pattern.

References

- [1] Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley.
- [2] Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3rd ed.). CreateSpace Independent Publishing Platform.
- [3] Hintjens, P. (2013). *ZeroMQ: Messaging for Many Applications*. O'Reilly Media.
- [4] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.